

A Multi-Agent Academic Research Assistant Design Proposal

System Overview and Design Foundations

The proposed system is a multi-agent academic research assistant that accepts a high-level research goal and returns a verified, explainable and structured report. Eight cooperating agents are coordinated within a single orchestration layer and communicate through a shared memory store, so that retrieved documents, intermediate results and decisions can be passed between stages (Figure 1). This blackboard-style shared context enables loosely coupled cooperation while keeping each agent's responsibility narrow and independently testable (Wooldridge, 2009). Underpinning these agents is a knowledge layer of free-tier academic APIs, including Semantic Scholar, arXiv, Crossref, PubMed and OpenAlex, together with a vector store (ChromaDB or FAISS) for semantic retrieval, which supplies the grounded evidence the agents draw on during the workflow.

Figure 1. High-Level System Architecture (Multi-Agent Academic Research Assistant)

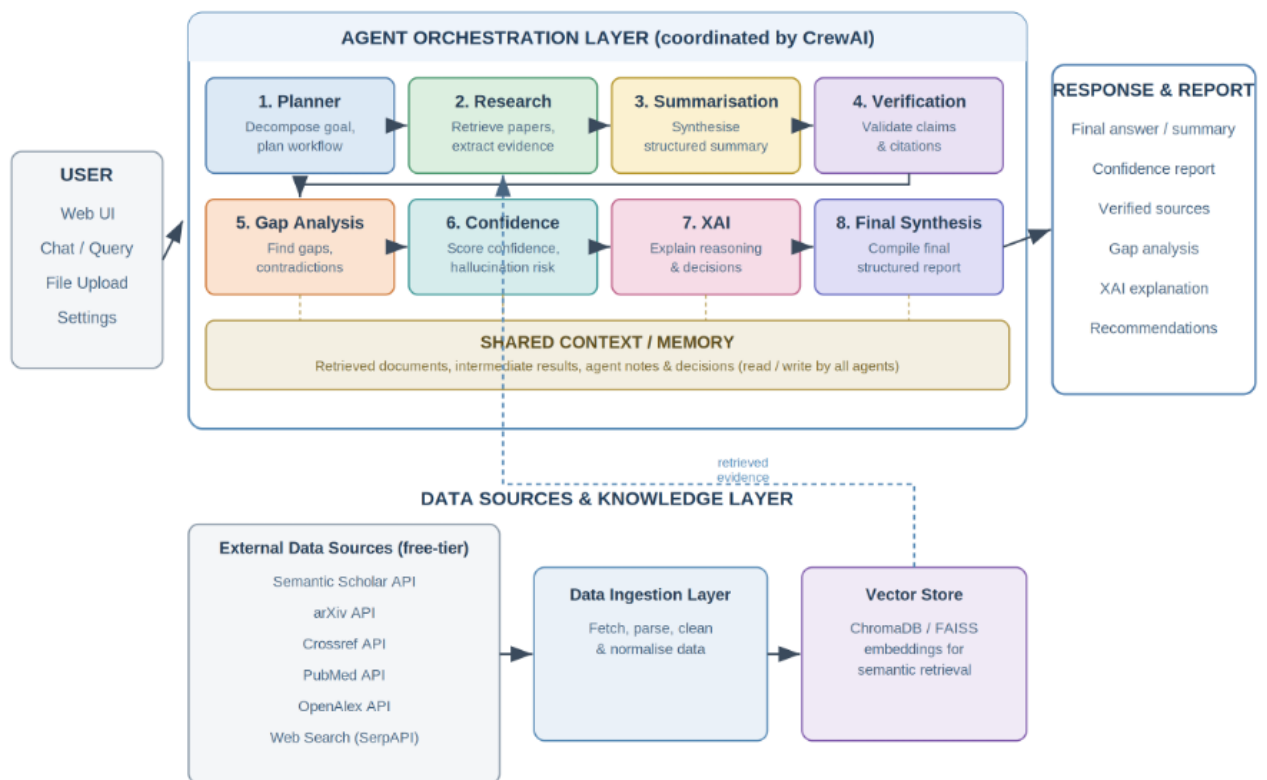


Figure 1. High-level system architecture showing the eight-agent orchestration layer, shared memory, knowledge layer and report output.

Agents and planning approach

A Planner agent decomposes the goal into ordered subtasks and selects the workflow, using LLM-supported reasoning in a plan, act and observe cycle similar to the ReAct pattern (Yao et al., 2023). The Research, Summarisation, Verification, Gap Analysis, Confidence, Explainable AI (XAI), and Final Synthesis agents then execute in turn. Planning is deliberative and goal-driven rather than purely reactive, reflecting the belief, desire, intention model in which agents act towards explicit goals (Rao and Georgeff, 1995). A feedback loop re-invokes the Planner when confidence is low or critical gaps remain, supporting iterative replanning (Figure 2).

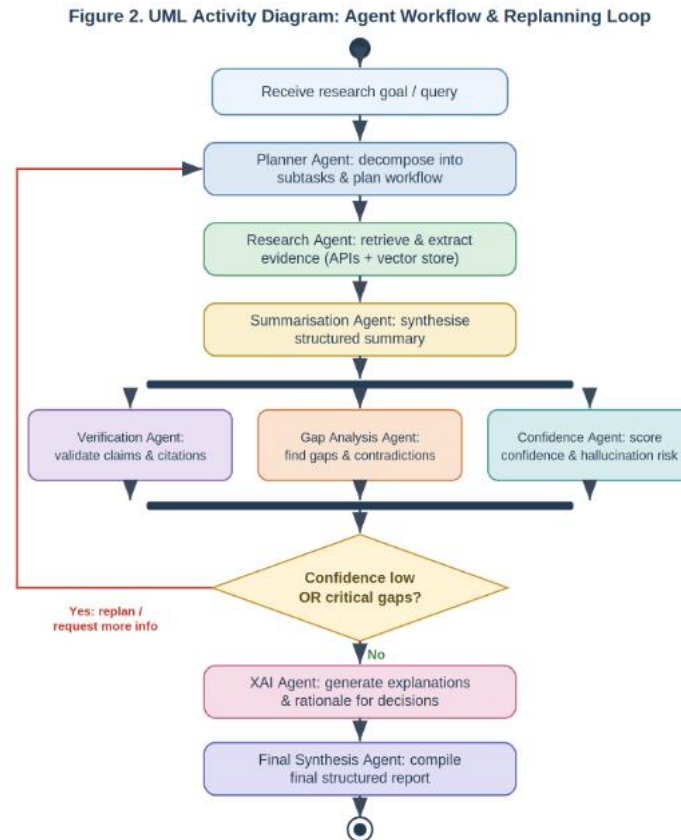


Figure 2. UML activity diagram of the agent workflow, including parallel verification and the confidence-driven replanning loop.

Development methodology and design rationale

The system was developed using an agent-oriented software engineering (AOSE) approach, taking the Tropos methodology (Bresciani et al., 2004) as its backbone. Tropos was chosen because, unlike object-oriented methods, it carries the same concepts, such as actors, goals, plans, and dependencies, through every phase from early requirements to implementation (Bresciani et al., 2004). These concepts map cleanly onto the deliberative layer of our hybrid, layered architecture (Wooldridge, 2009): Tropos goals become the agent's desires, plans become its intentions, and the actor's knowledge becomes its beliefs, while a reactive layer handles fast situated responses and a coordination layer manages social ability

between agents. Designing with Tropos therefore let us specify what each agent should want before committing to how it reasons.

To stay current, the methodology was extended along the lines of Asici et al. (2025), who revisit agent roles for LLM-enhanced multi-agent systems. Their work justified treating each agent's role and responsibilities as a first-class design artefact rather than hard-coding behaviour, keeping the design open to modern reasoning components alongside classical ones.

Framework and orchestration

Orchestration uses CrewAI, which provides role-based agents, task delegation, and shared context out of the box, reducing the engineering overhead of coordinating multiple LLM calls. Retrieval-augmented generation grounds every claim in retrieved literature to reduce hallucination (Lewis et al., 2020). CrewAI Flows also helps with staying within the memory requirements, by using ollama API calls to handle "Lazy Loading." Only the models actively required for the current stage (e.g., the Evaluators during the audit phase) are kept in the active VRAM/Unified Memory, while others are offloaded.

Development of the agents

The proposed system assumes stable internet access for academic API retrieval, sufficient free-tier API quotas for prototyping, access to hardware with 24 GB unified memory for local model deployment, and predominantly English-language source material.

Development follows a two-phase approach: rapid prototyping using Hugging Face cloud endpoints, followed by local deployment through Ollama. Development will be carried out in Python 3.11 using Visual Studio Code. Python 3.11 offers performance and memory-management improvements beneficial to asynchronous multi-agent systems (Python Software Foundation, 2025), while VS Code provides a flexible, cross-platform environment with demonstrated productivity benefits for software development (Pradista and Widodo, 2026). Unit testing will be performed in pytest, a widely adopted Python framework (Speth and Pretschner, 2024).

Following the findings of Chen et al. (2023), the system adopts a heterogeneous multi-model architecture deployed locally through Ollama, enabling tasks to be routed according to their computational and cognitive requirements. This approach improves both performance and resource efficiency compared with relying on a single general-purpose model. Reasoning-oriented tasks are assigned to Qwen models, selected for their documented reasoning performance (Qwen Team, 2025), while Llama models are used for text generation and synthesis. Model selection was constrained by the available 24GB of unified memory and optimised to balance reasoning performance, generation quality, and resource utilisation. Figure 3 sets out the resulting allocation of models to agents.

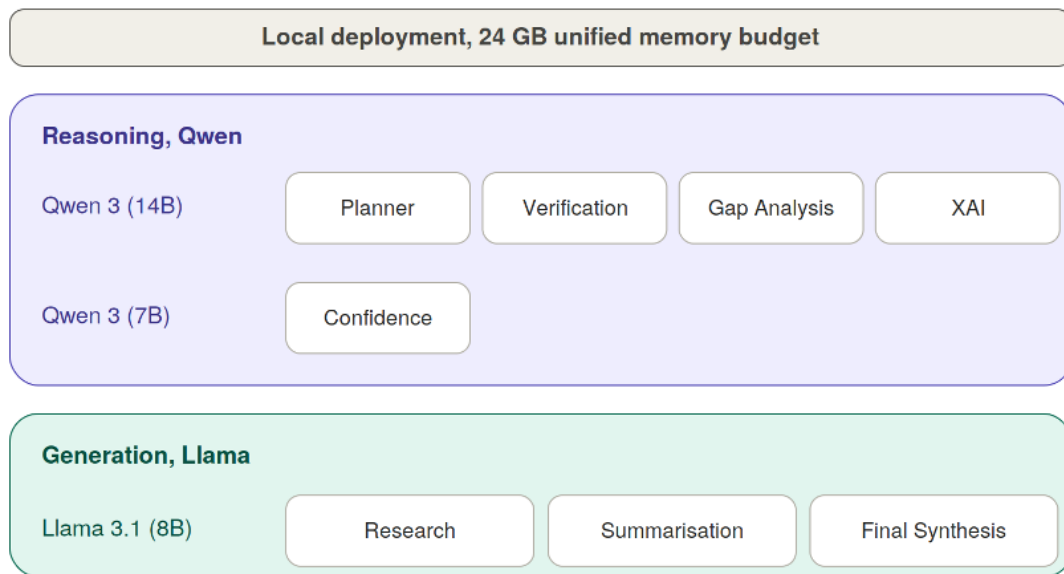


Figure 3. Allocation of local models to agents, showing the reasoning (Qwen) versus generation (Llama) split within the 24 GB unified-memory budget.

Anticipated Challenges

Developing this multi-agent system surfaces several interconnected challenges. The first is task decomposition reliability. The Planner is a single point of failure: any flawed or incomplete decomposition cascades to every downstream agent (Yao et al., 2023). We mitigate this with an iterative ReAct planning loop and a plan-validation mechanism, treating the Gap Analysis to Refine Plan cycle as a primary correction path.

A second challenge is hallucination and unsupported claims. Despite retrieval-augmented generation, LLMs may still produce partially unsupported statements during synthesis (Lewis et al., 2020; Ji et al., 2023). The difficulty is that our Verification and Confidence agents are themselves LLM-powered, so a fallible mechanism is being used to detect fallibility, and stated confidence scores are poorly calibrated and systematically overconfident. We therefore ground verification in claim-to-passage entailment against retrieved source spans rather than the model's own judgement and treat confidence as a relative signal that triggers refinement loops rather than an absolute probability shown to users. We acknowledge this risk can be reduced but not eliminated.

Noisy external data poses a third challenge. Academic APIs such as Semantic Scholar, Crossref, arXiv, and PubMed return preprints, retracted papers, and duplicate or inconsistent metadata. We apply semantic-similarity deduplication, rank sources by credibility heuristics that favour peer-reviewed work and require cross-source corroboration for key claims.

Explainability is a fourth concern. An LLM's stated reasoning may be post-hoc rationalisation, and explanations risk increasing user trust without increasing reliability, arguably worse than no explanation at all (Doshi-Velez and Kim, 2017). Rather than asking models to introspect, the XAI agent anchors explanations in traceable provenance: which source supported which claim, which agent flagged what, and what fed each confidence score. This makes explanations auditable rather than merely persuasive. Finally, managing agent cooperation is challenging because eight agents making LLM calls across refinement loops can trigger dozens of calls per query, and CrewAI's manager_llm makes routing and termination emergent judgements that risk runaway loops. We constrain this with a capable manager model, allow_delegation=False on workers, a tightly scoped manager prompt, and capped iterations, having the manager execute the Planner's plan rather than re-plan independently.

References

- Asici, T.Z., Gürcan, Ö. and Kardaş, G. (2025) 'Towards engineering LLM-enhanced multi-agent systems: a critical examination of roles', in S. Rodriguez, L. Feng and J.P. Müller (eds) *Engineering Multi-Agent Systems. EMAS 2025. Lecture Notes in Computer Science*, vol. 16407. Cham: Springer, pp. 200–220.
- Bresciani, P. *et al.* (2004) 'Tropos: an agent-oriented software development methodology', *Autonomous Agents and Multi-Agent Systems*, 8(3), pp. 203–236. Available at: <https://doi.org/10.1023/B:AGNT.0000018806.20944.ef> (Accessed: 24 May 2026).
- Chen, L., Zaharia, M., and Zou, J. (2023) 'FrugalGPT: how to use large language models while reducing cost and improving performance', *arXiv* [Preprint]. Available at: <https://arxiv.org/abs/2305.05176> (Accessed: 25 May 2026).
- Doshi-Velez, F., and Kim, B. (2017) 'Towards a rigorous science of interpretable machine learning', *arXiv* [Preprint]. Available at: <https://arxiv.org/abs/1702.08608> (Accessed: 30 May 2026).
- Duan, Z., and Wang, J. (2024) 'Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI', *arXiv* [Preprint]. Available at: <https://arxiv.org/abs/2411.18241> (Accessed: 31 May 2026).
- Ji, Z. *et al.* (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), Article 248. Available at: <https://doi.org/10.1145/3571730> (Accessed: 30 May 2026).
- Lewis, P. *et al.* (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474. Available at: <https://arxiv.org/abs/2005.11401> (Accessed: 30 May 2026).

- Pradista, N.N. and Widodo, S. (2026) ‘The effect of Visual Studio Code usability based on Nielsen’s attributes on the programming productivity of informatics students’, *JKTP: Jurnal Kajian Teknologi Pendidikan*, 9(2), pp. 243–253. Available at: <https://doi.org/10.17977/um038v9i22026p243-253> (Accessed: 31 May 2026).
- Python Software Foundation (2025) *What’s new in Python 3.11*. Available at: <https://docs.python.org/3.11/whatsnew/3.11.html> (Accessed: 31 May 2026).
- Qwen Team (2025) ‘Qwen3 technical report’, *arXiv* [Preprint]. Available at: <https://arxiv.org/abs/2505.09388> (Accessed: 30 May 2026).
- Rao, A.S. and Georgeff, M.P. (1995) ‘BDI agents: from theory to practice’, *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*. San Francisco, CA, 12–14 June. Menlo Park, CA: AAAI Press, pp. 312–319. Available at: <https://aaai.org/papers/icmas95-042-bdi-agents-from-theory-to-practice/> (Accessed: 16 May 2026).
- Speth, S. and Pretschner, A. (2024) ‘GeMTest: A General Metamorphic Testing Framework’, in *Proceedings of the IEEE/ACM International Conference on Software Engineering Workshops (ICSEW)*. Available at: <https://doi.org/10.1145/3639478.3640030> (Accessed: 07 June 2026).
- Wooldridge, M. (2009) *An Introduction to MultiAgent Systems*. 2nd edn. Chichester: John Wiley & Sons.
- Yao, S. *et al.* (2023) ‘ReAct: synergizing reasoning and acting in language models’, *International Conference on Learning Representations (ICLR 2023)*. Kigali, Rwanda, 1–5 May. Available at: <https://arxiv.org/abs/2210.03629> (Accessed: 16 May 2026)).